

Quasi-Cyclic Twisted McEliece Key Encapsulation Mechanism (QCTM)

Algorithm Specification

Submission Version for Commercial Cryptographic Scheme Solicitation

Algorithm type: public-key cryptographic algorithm, key encapsulation mechanism (KEM)

Algorithm name: Quasi-Cyclic Twisted McEliece Key Encapsulation Mechanism (QCTM)

Parameter instances: QCTM128, QCTM256, QCTM512

Submitter, contact person, intellectual-property statement, and signature information: see “Basic Algorithm Information”, “Statement of Intellectual Property Owners”, and “Statement of Owners of the Reference and Optimized Implementations” in the same submission package.

Implementation materials: see `Implementations/Reference_Implementation`, `Test_Vectors`, and `Supporting_Documentation` in the same submission package.

June 2026

Abstract

This document gives the algorithm specification for the Quasi-Cyclic Twisted McEliece Key Encapsulation Mechanism (QCTM). The scheme uses a Niederreiter-type syndrome ciphertext, and the underlying code family is given by the structure of quasi-cyclic twisted Goppa codes with affine translation. The public-key matrix is not a globally pure quasi-cyclic systematic matrix, but a hybrid systematic matrix consisting of a local quasi-cyclic main part and ordinary tail rows. The public key is stored as $(\psi(T_1), T_2)$, where T_1 is compressed in block-circulant form and T_2 is stored row by row. Decoding uses a decoding algorithm for twisted Goppa codes. The mathematical definitions, matrix generation, key generation, encapsulation, and decapsulation in this document are all based on the matrix construction adopted by the submitted implementation.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Scheme Type	4
2	Preliminaries	4
2.1	Notation	4
2.2	Goppa Codes, Twisted Goppa Codes, and Quasi-Cyclicity	4
3	Parameters	5
3.1	Parameter Requirements	5
3.2	Selected Parameters and Polynomials	6
3.2.1	Parameter Set QCTM128	6
3.2.2	Parameter Set QCTM256	6
3.2.3	Parameter Set QCTM512	6
3.3	Public-Key, Private-Key, and Ciphertext Sizes	6
4	One-Way Function	7
4.1	Goppa Polynomial Generation	7
4.2	Field Element Generation	8
4.3	Support Set Ordering Generation	9
4.4	Matrix Generation	10
4.5	Fixed-Weight Vector Generation	12
4.6	Encoding	12
4.7	Decoding	13
5	Key Encapsulation Mechanism	13
5.1	Key Generation	13
5.2	Encapsulation	14
5.3	Decapsulation	14
6	Security Considerations	14
6.1	Underlying Hard Problems	14
6.2	Known Attacks and Practical Security Levels	17
6.3	Structural Attack	19
6.4	Decapsulation Failure Analysis	20

7	Reference and Optimizational Implementations	20
7.1	Instantiation of the Irreducible Polynomials $f(z)$ and $F(y)$	20
7.2	Matrix Validation	21
7.3	Reference Implementation	21
7.4	Optimizational Implementation	21
8	Performance and Sizes	21
8.1	Algorithm Performance (Thousands of Clock Cycles)	21
8.2	Parameter Sizes (Bytes)	22
8.3	Known Answer Tests	22
9	Innovation Statement	22
10	Conclusion	22

1 Introduction

1.1 Motivation

The classical McEliece cryptosystem[30] has strong security, but its public key is excessively large. BIG QUAKE uses the quasi-cyclic structure of Goppa codes to compress the public key, which reduces the size but introduces potential attack risks. Therefore, we adopt a quasi-cyclic structure together with tail rows. This improves security relative to BIG QUAKE and significantly reduces the public-key size relative to the classical McEliece cryptosystem at the same security level.

The goal of QCTM is to use the following hybrid public-key form while retaining Niederreiter-type ciphertexts[32, 38] and the twisted Goppa decoding structure:

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}_{u \times n}, \quad u = m(t-1) + 1 \equiv r_0 \pmod{\ell},$$

where T_1 is an ℓ -block-circulant matrix and T_2 is an ordinary binary matrix with r_0 rows. Only T_1 is compressed in the public key, while T_2 is kept unchanged.

1.2 Scheme Type

This scheme is a key encapsulation mechanism. During encapsulation, a fixed-weight error vector e is sampled, the public syndrome $C = He^T$ is computed, and the session key is derived from (e, C) . During decapsulation, the twisted Goppa structure stored in the private key is used to decode and recover e . On success, the same session key is output; on failure, the random failure-branch vector s in the private key is used to output a pseudorandom key.

2 Preliminaries

2.1 Notation

This document uses the following notation.

Symbol	Meaning
q	2^m
m	extension degree of the binary extension field $\mathbb{F}_q/\mathbb{F}_2$
ℓ	odd prime satisfying $\ell \mid q-1$
ζ	element of order ℓ in $\mathbb{F}_q$
n	code length satisfying $\ell \mid n$
t_0	odd integer
t	degree of the Goppa polynomial, $t = \ell t_0$
k	dimension of the twisted Goppa code, $k = n - m(t-1) - 1$
w	encapsulation error weight, $w = (t-1)/2$
η	element of $\mathbb{F}_q^*$ initially sampled at random by KeyGen
g	monic irreducible polynomial of degree t in $\mathbb{F}_q[x]$
L	support set $\{\alpha_0, \dots, \alpha_{n-1}\}$ of the twisted Goppa code
Γ	binary twisted Goppa code $\Gamma(g, L, \eta)$
$\mathbf{H}$	hash function used for key derivation
$\mathbf{G}$	pseudorandom generator used for random seed expansion

Vectors are written as row vectors, and syndrome computation uses the column vector e^T . Row and column indices of binary matrices start from 0 by default in algorithm descriptions. When rows are described as “rows $1, \ell+1, 2\ell+1, \dots$ ”, the same set is being described using one-based numbering.

2.2 Goppa Codes, Twisted Goppa Codes, and Quasi-Cyclicity

Definition 2.1 (Binary Goppa code). [20, 21, 23] Let $g \in \mathbb{F}_q[x]$ be a monic irreducible polynomial of degree t , let $L = \{\alpha_0, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_q$, and assume that $g(\alpha_j) \neq 0$ for all $0 \leq j < n$. The binary

Goppa code is defined by

$$\Gamma(L, g) = \left\{ c = (c_0, \dots, c_{n-1}) \in \mathbb{F}_2^n : \sum_{j=0}^{n-1} \frac{c_j}{x - \alpha_j} \equiv 0 \pmod{g(x)} \right\}.$$

Definition 2.2 (Binary twisted Goppa code). Let $g \in \mathbb{F}_q[x]$, $L = \{\alpha_0, \dots, \alpha_{n-1}\} \subseteq \mathbb{F}_q$ satisfy $g(\alpha_j) \neq 0$, and let $\eta \in \mathbb{F}_q$. The binary twisted Goppa code is defined by

$$\Gamma(L, g, \eta) = \left\{ c = (c_0, \dots, c_{n-1}) \in \mathbb{F}_2^n : \sum_{j=0}^{n-1} c_j \left(\frac{1}{x - \alpha_j} - \eta \frac{\alpha_j^t}{g(\alpha_j)} \right) \equiv 0 \pmod{g(x)} \right\}.$$

When $\eta = 0$, $\Gamma(L, g, \eta)$ is the ordinary Goppa code $\Gamma(L, g)$.

Definition 2.3 (Quasi-cyclic code). Let $\mathcal{C} \subseteq \mathbb{F}_2^n$ be a linear code of length n . If $c = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}$, $\sigma_l(c) = (c_{-l}, c_{1-l}, \dots, c_{n-1-l}) \in \mathcal{C}$, and $d = \gcd(l, n)$, then $\mathcal{C}$ is a quasi-cyclic code of order d . When $l = 1$, $\mathcal{C}$ is a cyclic code.

Relevant literature on twisted Goppa codes and quasi-cyclic codes can be found in [5, 6, 7, 8, 9, 24, 25, 26, 27].

Definition 2.4 (ℓ -block-circulant matrix). Let ℓ be a positive integer and let M be a matrix. If M can be decomposed into several $\ell \times \ell$ circulant blocks, and each ℓ -circulant block has the form

$$\begin{pmatrix} a_0 & a_1 & \cdots & \cdots & a_{\ell-1} \\ a_{\ell-1} & a_0 & \cdots & \cdots & a_{\ell-2} \\ \vdots & \vdots & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & \vdots \\ a_1 & a_2 & \cdots & a_{\ell-1} & a_0 \end{pmatrix},$$

then M is called an ℓ -block-circulant matrix.

3 Parameters

3.1 Parameter Requirements

First, the QCTM scheme uses the following code-defining parameters:

1. a positive integer m and $q = 2^m$;
2. an odd prime ℓ satisfying $\ell \mid q - 1$;
3. a positive integer $n \leq q$ satisfying $\ell \mid n$;
4. an odd integer t_0 satisfying $m\ell t_0 < n$, $t = \ell t_0$, and $w = (t - 1)/2$;
5. an element ζ of order ℓ in $\mathbb{F}_q$;
6. a monic irreducible polynomial $f(z) \in \mathbb{F}_2[z]$ of degree m , used to represent $\mathbb{F}_q = \mathbb{F}_2[z]/f(z)$. See Section 7 for the concrete form.
7. a monic irreducible polynomial $F(y) \in \mathbb{F}_q[y]$ of degree t_0 , used to represent $\mathbb{F}_{q^{t_0}} = \mathbb{F}_q[y]/F(y)$. See Section 7 for the concrete form.

Second, the QCTM scheme uses the following symmetric cryptographic parameters and components:

1. a positive integer $\lambda \geq 256$. For QCTM128 and QCTM256, set $\lambda = 256$; for QCTM512, set $\lambda = 512$.
2. a cryptographic hash function $\mathbf{H}$ with output length λ . It may be instantiated by `CryptHash` provided by the algorithm solicitation platform.
3. a pseudorandom number generator $\mathbf{G}$, which expands a bit string of length λ into a bit string of length $n + \sigma_2(q - 1) + \sigma_1(t_0 + 1) + \lambda$, where $\sigma_1 \geq m$ and $\sigma_2 \geq 2m$. It may be instantiated by `get_random_number` provided by the algorithm solicitation platform.

3.2 Selected Parameters and Polynomials

To meet the structural design requirements of the quasi-cyclic twisted Goppa codes needed by QCTM, each submitted parameter instance must specify both its KEM parameters and the two irreducible polynomials

$$f(z) \in \mathbb{F}_2[z] \text{ and } F(y) \in \mathbb{F}_q[y]$$

where $\mathbb{F}_q \triangleq \mathbb{F}_2[z]/(f(z))$. Considering implementation efficiency, we choose the following three submitted parameter instances.

3.2.1 Parameter Set QCTM128

This KEM has parameters $m = 18$, $n = 10070$, $\ell = 19$, $t_0 = 15$, $t = 285$, $k = 4957$, and $w = 142$. The public-key length is 167,919 bytes, the private-key length is 23,299 bytes, and the ciphertext length is 640 bytes. The field polynomials are $f(z) = z^{18} + z^3 + 1$ and $F(y) = y^{15} + y^7 + z^7$.

3.2.2 Parameter Set QCTM256

This KEM has parameters $m = 18$, $n = 19000$, $\ell = 19$, $t_0 = 27$, $t = 513$, $k = 9783$, and $w = 256$. The public-key length is 595,541 bytes, the private-key length is 43,905 bytes, and the ciphertext length is 1,153 bytes. The field polynomials are $f(z) = z^{18} + z^7 + 1$ and $F(y) = y^{27} + z$.

3.2.3 Parameter Set QCTM512

This KEM has parameters $m = 18$, $n = 38000$, $\ell = 19$, $t_0 = 53$, $t = 1007$, $k = 19891$, and $w = 503$. The public-key length is 2,374,489 bytes, the private-key length is 87,766 bytes, and the ciphertext length is 2,264 bytes. The field polynomials are $f(z) = z^{18} + z^3 + 1$ and $F(y) = y^{53} + y^6 + z$.

3.3 Public-Key, Private-Key, and Ciphertext Sizes

Let

$$u = m(t - 1) + 1 \equiv r_0 \pmod{\ell}, \quad 0 \leq r_0 < \ell.$$

The systematic public matrix is

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}_{u \times n},$$

where T_1 is an $(u - r_0) \times (n - u + r_0)$ ℓ -block-circulant matrix, and T_2 is an ordinary binary matrix of size $r_0 \times (n - u + r_0)$. Therefore, the number of payload bits in the public key is

$$\text{pkbits} = \frac{u - r_0}{\ell}(n - u + r_0) + r_0(n - u + r_0),$$

and the public-key size in bytes is

$$\text{pkbytes} = \left\lceil \frac{1}{8} \left(\frac{m(t - 1) + 1 - r_0}{\ell} (n - (m(t - 1) + 1) + r_0) + r_0(n - (m(t - 1) + 1) + r_0) \right) \right\rceil.$$

The private-key size in bytes is

$$\text{skbytes} = \left\lceil \frac{1}{8} (n + t)m \right\rceil.$$

The ciphertext size in bytes is

$$\text{ctbytes} = \left\lceil \frac{1}{8} (m(t - 1) + 1) \right\rceil.$$

Note that the parameter sets of QCTM128, QCTM256 and QCTM512, the related public-key, private-key and ciphertext sizes are given in Subsection 8.2.

4 One-Way Function

4.1 Goppa Polynomial Generation

Algorithm 1: GoppaPolynomial

Input: Bit string $d_0, d_1, \dots, d_{\sigma_1(t_0+1)-1}$ of length $\sigma_1(t_0 + 1)$
Output: $\perp$, or (η, g) , where $\eta \in \mathbb{F}_q^*$ and $g \in \mathbb{F}_q[x]$ is a monic irreducible polynomial of degree $t = \ell t_0$

- 1 Split the input bit string into $t_0 + 1$ blocks, each of σ_1 bits, and use only the first m bits of each block;
- 2 **for** $j = 0$ **to** $t_0 - 1$ **do**
- 3 $\beta_j \leftarrow \sum_{i=0}^{m-1} d_{\sigma_1 j + i} z^i \in \mathbb{F}_q$;
- 4 $\eta \leftarrow \sum_{i=0}^{m-1} d_{\sigma_1 t_0 + i} z^i \in \mathbb{F}_q$;
- 5 **if** $\eta = 0$ **then**
- 6 $\perp$ **return** $\perp$;
- 7 $\beta \leftarrow \beta_0 + \beta_1 y + \dots + \beta_{t_0-1} y^{t_0-1} \in \mathbb{F}_q[y]/F(y)$;
- 8 Compute the minimal polynomial M of β over $\mathbb{F}_q$ (of degree t_0);
- 9 **if** $\deg M \neq t_0$ **then**
- 10 $\perp$ **return** $\perp$;
- 11 Compute $\text{ord}(\beta)$ and check whether its ℓ -adic valuation is equal to that of $q^{t_0} - 1$;
- 12 **if they are not equal then**
- 13 $\perp$ **return** $\perp$;
- 14 $g(x) \leftarrow M\left(\left(x - \frac{1}{\eta}\right)^\ell\right) = g_0 + g_1 x + \dots + g_{t-1} x^{t-1} + x^t$, which is irreducible over $\mathbb{F}_q$;
- 15 **return** (η, g) ;

Comments on Algorithm 1.

Input A bit string $d_0, \dots, d_{\sigma_1(t_0+1)-1}$ of length $\sigma_1(t_0 + 1)$, $\sigma_1 \geq m$.

Output $\perp$, or $\eta \in \mathbb{F}_q^*$ and a monic irreducible polynomial $g \in \mathbb{F}_q[x]$ of degree $t = \ell t_0$.

Procedure

1. Divide the bit string of length $\sigma_1(t_0 + 1)$ into $t_0 + 1$ blocks. For each $j \in \{0, \dots, t_0 - 1\}$, define

$$\beta_j = \sum_{i=0}^{m-1} d_{\sigma_1 j + i} z^i \in \mathbb{F}_q,$$

$$\eta = \sum_{i=0}^{m-1} d_{\sigma_1 t_0 + i} z^i \in \mathbb{F}_q.$$

Only the first m bits of each block of σ_1 bits are used. If $\eta = 0$, return $\perp$.

2. Define

$$\beta = \beta_0 + \beta_1 y + \dots + \beta_{t_0-1} y^{t_0-1} \in \mathbb{F}_q[y]/F(y).$$

3. Compute the minimal polynomial M of β over $\mathbb{F}_q$, of degree t_0 . If $N_q(t_0)$ is the number of irreducible polynomials of degree t_0 in $\mathbb{F}_q[x]$ (see [28]), the success probability is $t_0 N_q(t_0) / q^{t_0}$. When $t_0 \geq 16$, this probability exceeds 99%. If the degree of M is not t_0 , return $\perp$.
4. Compute $\text{ord}(\beta)$, and require the ℓ -adic valuation of $\text{ord}(\beta)$ to be equal to that of $q^{t_0} - 1$. If this condition is not satisfied, return $\perp$.
5. $M(x^\ell)$ is irreducible over $\mathbb{F}_q$. Set

$$g(x) = M\left(\left(x - \frac{1}{\eta}\right)^\ell\right) = g_0 + g_1 x + \dots + g_{t-1} x^{t-1} + x^t.$$

Remark: $M(x)$ is irreducible over $\mathbb{F}_q$. Moreover, $M(x^\ell)$ is irreducible over $\mathbb{F}_q$ if and only if $x^\ell - \beta$ is irreducible over $\mathbb{F}_{q^{t_0}}$ if and only if the ℓ -adic valuation of $\text{ord}(\beta)$ is equal to that of $q^{t_0} - 1$ (see [28]) if and only if $\beta \notin (\mathbb{F}_{q^{t_0}})^\ell$. Therefore, the success probability is $1 - 1/\ell$. When $\ell \geq 13$, the success probability exceeds 92.3%.

4.2 Field Element Generation

Algorithm 2: FieldOrdering

Input: Bit string $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ of length $\sigma_2(q-1)$

Output: $\perp$, or a sequence $(a_1, \dots, a_{q-1})$ of $q-1$ distinct nonzero elements in $\mathbb{F}_q$

- 1 Divide the bit string of length $\sigma_2(q-1)$ into $q-1$ blocks, and use each block of σ_2 bits to form a nonnegative integer:

$$h_i = b_{\sigma_2(i-1)} + 2b_{\sigma_2(i-1)+1} + \dots + 2^{\sigma_2-1}b_{\sigma_2 i-1};$$

- 2 **if** $h_1, \dots, h_{q-1}$ are not pairwise distinct **then**

- 3 **return** $\perp$;

- 4 Sort (h_i, i) lexicographically, with smaller values first, and write the result as $(h_{\pi(i)}, \pi(i))$;

- 5 Define

$$a_i = \sum_{j=0}^{m-1} \pi(i)_j z^{m-1-j},$$

where $\pi(i) = \sum_{j=0}^{m-1} \pi(i)_j 2^j$;

- 6 **return** $(a_1, \dots, a_{q-1})$;
-

Comments on Algorithm 2.

Input A bit string $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ of length $\sigma_2(q-1)$, $\sigma_2 \geq 2m$.

Output $\perp$, or a sequence $(a_1, \dots, a_{q-1})$ of $q-1$ distinct nonzero elements in $\mathbb{F}_q$.

Procedure

1. Divide the bit string of length $\sigma_2(q-1)$ into $q-1$ blocks. Each block of σ_2 bits defines a nonnegative integer

$$h_i = b_{\sigma_2(i-1)} + 2b_{\sigma_2(i-1)+1} + \dots + 2^{\sigma_2-1}b_{\sigma_2 i-1},$$

where each $0 \leq h_i < 2^{\sigma_2}$.

2. If $h_1, \dots, h_{q-1}$ are not pairwise distinct, return $\perp$ (It is highly probable that all pairs are distinct).
3. Sort (h_i, i) lexicographically, with smaller values first, and write the result as $(h_{\pi(i)}, \pi(i))$.
4. Define

$$a_i = \sum_{j=0}^{m-1} \pi(i)_j z^{m-1-j},$$

where $\pi(i) = \sum_{j=0}^{m-1} \pi(i)_j 2^j$.

5. Output $(a_1, \dots, a_{q-1})$, where each $a_i \neq 0$.

4.3 Support Set Ordering Generation

Algorithm 3: DefinedSet

Input: $\eta \in \mathbb{F}_q^*$ and a bit string $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ of length $\sigma_2(q-1)$
Output: $\perp$, or a set $L = \{\alpha_0, \dots, \alpha_{n-1}\}$ of n elements in $\mathbb{F}_q$

```

1  $(a_1, \dots, a_{q-1}) \leftarrow \text{FIELDORDERING}(b_0, \dots, b_{\sigma_2(q-1)-1});$ 
2 if FieldOrdering fails then
3   return  $\perp$ ;
4  $A \leftarrow \begin{pmatrix} \zeta & \frac{\zeta+1}{\eta} \\ 0 & 1 \end{pmatrix};$ 
5  $\mathcal{O} \leftarrow [];$ 
6 for  $i = 1$  to  $q-1$  do
7   if  $a_i + \frac{1}{\eta}$  is not contained in an orbit already stored in  $\mathcal{O}$  then
8      $O_i \leftarrow \left\{ a_i + \frac{1}{\eta}, \zeta a_i + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_i + \frac{1}{\eta} \right\}$ 
9     Append  $O_i$  to  $\mathcal{O}$ ;
10  if  $|\mathcal{O}| = n/\ell$  then
11    break;
12 Form  $L$  by concatenating the orbits in  $\mathcal{O}$  in their stored order;
13
14 
$$L = \left\{ a_1 + \frac{1}{\eta}, \zeta a_1 + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_1 + \frac{1}{\eta}; \dots; a_{\frac{n}{\ell}} + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_{\frac{n}{\ell}} + \frac{1}{\eta} \right\};$$

15 return  $L$ ;
```

Comments on Algorithm 3.

Input $\eta \in \mathbb{F}_q^*$ and a bit string $b_0, b_1, \dots, b_{\sigma_2(q-1)-1}$ of length $\sigma_2(q-1)$, $\sigma_2 \geq 2m$.

Output $\perp$, or a set $L = \{\alpha_0, \dots, \alpha_{n-1}\}$ of n elements in $\mathbb{F}_q$.

Procedure

1. Call FieldOrdering on the input bit string to obtain $(a_1, \dots, a_{q-1})$. If the call fails, return $\perp$.
2. Select, in order, representatives $a_i + \frac{1}{\eta}$ that have not yet appeared in an already constructed multiplicative orbit. Use the affine transformation

$$A = \begin{pmatrix} \zeta & \frac{\zeta+1}{\eta} \\ 0 & 1 \end{pmatrix},$$

and construct the affine-translation orbit

$$O_i = \left\{ a_i + \frac{1}{\eta}, \zeta a_i + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_i + \frac{1}{\eta} \right\}.$$

3. Take the first $\frac{n}{\ell}$ such orbits and output them in orbit order:

$$L = \left\{ a_1 + \frac{1}{\eta}, \zeta a_1 + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_1 + \frac{1}{\eta}; \dots; a_{\frac{n}{\ell}} + \frac{1}{\eta}, \dots, \zeta^{\ell-1} a_{\frac{n}{\ell}} + \frac{1}{\eta} \right\}.$$

4.4 Matrix Generation

Algorithm 4: MatGen

Input: $\Gamma = (g, L, \eta)$, where $g \in \mathbb{F}_q[x]$ is a monic irreducible polynomial of degree $t = \ell t_0$,
 $L = \{\alpha_0, \dots, \alpha_{n-1}\}$, and $\eta \in \mathbb{F}_q^*$

Output: $\perp$, or $(\psi(T_1), T_2)$ and $\Gamma' = (g, L', \eta)$

- 1 Compute the $t \times n$ matrix $H_T = (h_{i,j})$ over $\mathbb{F}_q$ and apply elementary row operations:

$$H_T = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ g(\alpha_0) & g(\alpha_1) & \cdots & g(\alpha_{n-1}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{\alpha_0^{t-1} + \eta \alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta \alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta \alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix} \sim \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ g(\alpha_0) & g(\alpha_1) & \cdots & g(\alpha_{n-1}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ 1 & 1 & \cdots & 1 \end{pmatrix};$$

- 2 Write every element $h_{i,j}$ in the first $t-1$ rows of H_T as

$$u_0 + u_1 z + \cdots + u_{m-1} z^{m-1}.$$

Replace $h_{i,j}$ by the binary column vector $(u_0, u_1, \dots, u_{m-1})$, and do not expand the all-one last row. This gives

$$\hat{H}_T \in \mathbb{F}_2^{(m(t-1)+1) \times n};$$

- 3 $u \leftarrow m(t-1) + 1$;
4 Use $\hat{H}_T$ to construct an ℓ -block-circulant matrix $H_1 \in \mathbb{F}_2^{\ell u \times n}$ by applying block-cyclic shifts to each row of $\hat{H}_T$;
5 Apply ℓ -circulant-block Gaussian elimination to H_1 ;
6 **if** *successful* **then**
7 Obtain a block permutation τ of the columns and an invertible matrix Q_0 such that
$$Q_0 \tau(H_1) = \begin{pmatrix} H \\ 0 \end{pmatrix},$$
where
$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}, \quad r_0 \equiv u \pmod{\ell};$$
8 Extract rows $1, \ell+1, 2\ell+1, \dots, (u-r_0)-\ell+1$ from T_1 to form
$$\psi(T_1) \in \mathbb{F}_2^{\frac{u-r_0}{\ell} \times (n-u+r_0)};$$
9 $L' \leftarrow \tau(L)$;
 // There exists an invertible $u \times u$ matrix Q and the same column
 permutation τ such that $Q\tau(\hat{H}_T) = H$
10 **return** $(\psi(T_1), T_2)$, $\Gamma' = (g, L', \eta)$;
11 **else**
12 **return** $\perp$;
-

Comments on Algorithm 4.

Input $\Gamma = (g, L, \eta)$, where $g \in \mathbb{F}_q[x]$ is a monic irreducible polynomial of degree $t = \ell t_0$, $L = \{\alpha_0, \dots, \alpha_{n-1}\}$, and $\eta \in \mathbb{F}_q^*$.

Output $\perp$, or $(\psi(T_1), T_2)$ and $\Gamma' = (g, L', \eta)$.

Procedure

1. Compute the $t \times n$ matrix over $\mathbb{F}_q$

$$H_T = (h_{i,j}) = \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{\alpha_0}{g(\alpha_0)} & \frac{\alpha_1}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{\alpha_0^{t-1} + \eta \alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta \alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta \alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix}.$$

After elementary row operations on H_T , one obtains

$$H_T \sim \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{\alpha_0}{g(\alpha_0)} & \frac{\alpha_1}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}}{g(\alpha_{n-1})} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ 1 & 1 & \cdots & 1 \end{pmatrix}.$$

Note that the last row is an all-one row.

2. Write every element $h_{i,j}$ in the first $t-1$ rows of H_T as

$$u_0 + u_1 z + \cdots + u_{m-1} z^{m-1}.$$

Then replace $h_{i,j}$ by the binary column vector $(u_0, u_1, \dots, u_{m-1})$. The all-one last row is not expanded. This gives

$$\hat{H}_T \in \mathbb{F}_2^{(m(t-1)+1) \times n}.$$

With high probability, $\hat{H}_T$ has rank $m(t-1) + 1$.

3. Set $u = m(t-1) + 1$. Use $\hat{H}_T$ to construct an ℓ -block-circulant matrix $H_1 \in \mathbb{F}_2^{\ell u \times n}$ by applying block-cyclic shifts to each row of $\hat{H}_T$.
4. Apply ℓ -circulant-block Gaussian elimination to H_1 . If successful, obtain a block permutation τ of the columns and an invertible matrix Q_0 such that

$$Q_0 \tau(H_1) = \begin{pmatrix} H \\ 0 \end{pmatrix},$$

where

$$\hat{H}_T \sim H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix}, \quad r_0 \equiv u \pmod{\ell}.$$

Here T_1 is an $(u-r_0) \times (n-u+r_0)$ ℓ -block-circulant matrix, and T_2 is an $r_0 \times (n-u+r_0)$ full-row-rank matrix. In current tests, 5000 trials all succeeded; in the sense used by BIG QUAKE, this is a high-probability event. Extract rows $1, \ell+1, 2\ell+1, \dots, (u-r_0)-\ell+1$ from T_1 to form

$$\psi(T_1) \in \mathbb{F}_2^{\frac{u-r_0}{\ell} \times (n-u+r_0)}.$$

5. Output:

$$(\psi(T_1), T_2), \Gamma' = (g, L', \eta),$$

where $L' \leftarrow \tau(L)$. There exists an invertible $u \times u$ matrix Q and the same column permutation τ such that

$$Q\tau(\hat{H}_T) = H.$$

If the procedure fails, return $\perp$.

Remark: The public matrix output by MatGen is a hybrid systematic matrix. Only T_1 is compressed as an ℓ -block-circulant matrix; the r_0 rows of T_2 are stored as an ordinary binary matrix. The auxiliary matrices Q_0 , Q , and τ produced during systematic reduction do not need to be stored.

4.5 Fixed-Weight Vector Generation

Algorithm 5: FixedWeight

Input: Parameter set and internal randomness

Output: A vector $e \in \mathbb{F}_2^n$ of weight $w = (t-1)/2$

```

1 Choose  $\rho$ : if  $n = q$ , set  $\rho = w$ ; if  $q/2 \leq n < q$ , set  $\rho = 2w$ ; and so on;
2 repeat
3   Generate  $\sigma_1\rho$  uniformly random bits  $b_0, \dots, b_{\sigma_1\rho-1}$ ;
4   for  $j = 0$  to  $\rho - 1$  do
5      $d_j \leftarrow \sum_{i=0}^{m-1} b_{\sigma_1 j+i} 2^i$ ;
6   Let  $D$  be the ordered list of pairwise distinct elements among  $d_0, \dots, d_{\rho-1}$  that belong to
      $\{0, \dots, n-1\}$ ;
7 until  $|D| \geq w$ ;
8 Initialize  $e \leftarrow (0, \dots, 0) \in \mathbb{F}_2^n$ ;
9 for the first  $w$  positions  $j$  in  $D$  do
10   Set  $e_j \leftarrow 1$ ;
11 return  $e$ ;
```

4.6 Encoding

Algorithm 6: Encode

Input: A vector $e \in \mathbb{F}_2^n$ of weight $w = (t-1)/2$, and public key $(\psi(T_1), T_2)$

Output: Ciphertext syndrome $C \in \mathbb{F}_2^u$

1 Recover the ℓ -block-circulant matrix T_1 from $\psi(T_1)$, and define

$$H = \begin{pmatrix} I_{u-r_0} & T_1 \\ 0 & T_2 \end{pmatrix};$$

2 return

$$C = He^T \in \mathbb{F}_2^u;$$

4.7 Decoding

Algorithm 7: Decode

Input: Ciphertext $C \in \mathbb{F}_2^u$, private key $\Gamma' = (g, L', \eta)$, and public key $(\psi(T_1), T_2)$
Output: A vector $e \in \mathbb{F}_2^n$ of weight $w = (t-1)/2$, or $\perp$

- 1 Re-execute Steps 1 and 2 of MatGen using Γ' to obtain $\hat{H}_T$;
- 2 Execute Step 1 of Encode using $(\psi(T_1), T_2)$ to recover H ;
- 3 Compute Q^{-1} such that $\hat{H}_T = Q^{-1}H$;
- 4 Compute $\hat{H}_T e^T = Q^{-1}H e^T = Q^{-1}C$, using $C = H e^T$;
- 5 Recover the result as elements of $\mathbb{F}_q$ using the basis $1, z, \dots, z^{m-1}$;
- 6 Recover subsequent coordinates in order, keeping the coordinate corresponding to the last row as the unit coordinate in $\mathbb{F}_2$;
- 7 Set
$$S = H_T e^T;$$
- 8 $e \leftarrow \text{TWISTEDGOPPADECODE}(S, g, L', \eta)$ using the decoding algorithm of [36, 37];
- 9 **if** e does not exist or $\text{wt}(e) \neq w$ **then**
- 10 **return** $\perp$;
- 11 **return** e ;

Remark: Repeated tests show that the minimum distance of the twisted Goppa code used in this scheme is $t+1$. This decoding algorithm is optimal, and it is different from the Patterson algorithm [33] commonly used for Goppa codes.

5 Key Encapsulation Mechanism

5.1 Key Generation

Algorithm 8: KeyGen

Input: Parameter set and internal randomness
Output: Public key pk and private key sk

- 1 Sample a λ -bit seed δ uniformly at random;
- 2 **return** SeededKeyGen(δ);

Here, the algorithm SeededKeyGen is given below.

Algorithm 9: SeededKeyGen

Input: Seed δ
Output: Public key pk and secret key sk

- 1 Compute $E = \mathbf{G}(\delta)$, obtaining the
$$n + \sigma_2(q-1) + \sigma_1(t_0+1) + \lambda$$
bit string;
- 2 Let δ' be the last λ bits of E , and let s be the first n bits of E ;
- 3 Let ρ be the next $\sigma_1(t_0+1)$ bits of E ;
- 4 $(\eta, g) \leftarrow \text{GoppaPolynomial}(\rho)$;
- 5 **if** $(\eta, g) = \perp$ **then**
- 6 $\delta \leftarrow \delta'$;
- 7 restart;
- 8 Let ρ_L be the next $\sigma_2(q-1)$ bits of E ;
- 9 $L \leftarrow \text{DefinedSet}(\rho_L)$;
- 10 **if** $L = \perp$ **then**
- 11 $\delta \leftarrow \delta'$;
- 12 restart;

Algorithm 9: SeededKeyGen (continued)

```
13 Let  $\Gamma = (g, L)$ ;  
14  $(\psi(T_1), T_2, \Gamma') \leftarrow \text{MatGen}(\Gamma)$ ;  
15 if  $(\psi(T_1), T_2, \Gamma') = \perp$  then  
16    $\delta \leftarrow \delta'$ ;  
17   restart;  
18 Output the public key pk and the secret key sk as follows:
```

$$\text{pk} = (\psi(T_1), T_2), \quad \text{sk} = (\delta, \Gamma', s);$$

5.2 Encapsulation

Algorithm 10: Encaps

```
Input: Public key  $\text{pk} = (\psi(T_1), T_2)$   
Output: Ciphertext  $C$  and session key  $K$   
1  $e \leftarrow \text{FIXEDWEIGHT}$ , where  $e \in \mathbb{F}_2^n$  and  $\text{wt}(e) = w = (t-1)/2$ ;  
2  $C \leftarrow \text{ENCODE}(e, \psi(T_1), T_2)$ ;  
3  $K \leftarrow \mathbf{H}(1, e, C)$ ;  
4 return  $(C, K)$ ;
```

5.3 Decapsulation

Algorithm 11: Decaps

```
Input: Ciphertext  $C$  and secret key  $\text{sk} = (\delta, \Gamma', s)$   
Output: Session key  $K$   
1  $b \leftarrow 1$ ;  
2  $e \leftarrow \text{DECODE}(C, \Gamma')$ ;  
3 if  $e = \perp$  then  
4    $e \leftarrow s$ ;  
5    $b \leftarrow 0$ ;  
6  $K \leftarrow \mathbf{H}(b, e, C)$ ;  
7 return  $K$ ;
```

6 Security Considerations

The design of the QCTM scheme draws on both Classic McEliece[11] and BIG QUAKE[1]. Compared with Classic McEliece, the difference is that a part of the public-key matrix is compressed using a quasi-group-ring structure, so the overall public-key size is greatly reduced. Compared with BIG QUAKE, the difference is that the scheme does not use a quasi-cyclic structure for the whole public-key matrix; instead, it introduces a twisted term in the last row of the Goppa parity-check matrix, so the overall public-key size increases only slightly. The intended effect is to strike a balance between security and efficiency, especially public-key size: the introduction of the twisted term and the disruption of the global quasi-cyclic structure are intended to avoid possible attacks using invariant subcodes of the quasi-cyclic structure, while the preservation of local quasi-cyclic structure still enables substantial public-key compression. Overall, the security of the QCTM scheme lies between that of Classic McEliece and BIG QUAKE.

We analyze the scheme from three aspects: underlying hard problems, basis for practical security estimation, and parameter selection.

6.1 Underlying Hard Problems

The security of QCTM relies on the hardness, under the listed parameters, of the following problems:

- given a public binary parity-check matrix and a syndrome, find an error vector of weight w ;
- recover the private twisted Goppa structure Γ' from the public hybrid systematic matrix $(\psi(T_1), T_2)$;
- distinguish the public matrix generated by KeyGen from a random hybrid systematic matrix of the same size;
- rely on the random-oracle or pseudorandomness assumptions for the hash function, XOF, or pseudorandom generator used by the scheme.

The security of this scheme is based on several fundamental hard problems in coding theory. Because the scheme uses twisted Goppa codes with hidden algebraic structure and further introduces quasi-cyclic structure, its security depends not only on the hardness of the generic syndrome decoding problem, but also on private code-structure hiding, the quasi-cyclic syndrome decoding problem, and public-key indistinguishability assumptions[3].

Definition 1: Syndrome decoding problem $\text{SD}(n, k, w)$. Given positive integers n, k, w , where $0 < k < n$, let

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

be a public binary parity-check matrix, and let

$$s \in \mathbb{F}_2^{n-k}$$

be a public syndrome. The syndrome decoding problem $\text{SD}(n, k, w)$ asks one to find an error vector

$$e \in \mathbb{F}_2^n$$

such that

$$\text{wt}(e) = w, \quad He^T = s,$$

where $\text{wt}(e)$ denotes the Hamming weight of e .

This problem characterizes the difficulty of recovering a low-weight error vector when the attacker knows only the public parity-check matrix and the syndrome. In Niederreiter-type or McEliece-type code-based cryptography, the ciphertext or encapsulation value can usually be represented as the syndrome corresponding to a low-weight error vector. Thus, if an attacker can efficiently solve $\text{SD}(n, k, w)$, the attacker can directly recover the encapsulation error vector and break confidentiality. The main known attacks on this problem are information set decoding attacks, including Prange, Stern, Dumer, Ball-Collision, BJMM, and May-Ozerov. Therefore, the classical security strength of a parameter set (n, k, w) is usually determined by the lowest complexity among these attacks.

Definition 2: Twisted Goppa private-structure recovery problem. Let Γ' denote the twisted Goppa private structure used by the key generation algorithm. This structure includes the support set, Goppa polynomial, twisting parameter, permutation transformation, and secret algebraic information related to efficient decoding. The key generation algorithm constructs a private code with an efficient decoding algorithm from Γ' , and then generates the public hybrid systematic matrix

$$H_{\text{pub}} = (\psi(T_1), T_2)$$

through hiding transformations, systematic reduction, and binary expansion. The twisted Goppa private-structure recovery problem is the following: given the public matrix

$$H_{\text{pub}} = (\psi(T_1), T_2),$$

recover the original private structure Γ' , or recover an equivalent private structure Γ'' such that the code defined by Γ'' is equivalent to the code corresponding to the public matrix and supports efficient decoding.

This problem corresponds to structure-recovery or key-recovery attacks in code-based cryptography. Unlike the syndrome decoding problem, the goal of a structure-recovery attack is not to solve

for the error vector of one encapsulation, but to recover the hidden private code structure from the public matrix. Once an attacker obtains Γ' or an equivalent structure, the attacker can efficiently decode arbitrary syndromes just like the legitimate decapsulator. Thus, the hardness of recovering the private twisted Goppa structure ensures that the public matrix does not leak algebraic information sufficient to recover the private decoding capability.

Definition 3: Search ℓ -quasi-cyclic syndrome decoding problem ℓ -QCSD(n, k, w). Given positive integers n, k, w, ℓ , where $0 < k < n$, let

$$H \in \mathbb{F}_2^{(n-k) \times n}$$

be a public binary parity-check matrix of a systematic ℓ -quasi-cyclic code, and let

$$s \in \mathbb{F}_2^{n-k}$$

be a public syndrome. The search ℓ -quasi-cyclic syndrome decoding problem, denoted

$$\ell\text{-QCSD}(n, k, w),$$

asks one to find an error vector

$$e \in \mathbb{F}_2^n$$

such that

$$\text{wt}(e) = w, \quad He^T = s.$$

Compared with the ordinary syndrome decoding problem, the public parity-check matrix in the ℓ -quasi-cyclic syndrome decoding problem has additional quasi-cyclic structure. The quasi-cyclic structure can significantly compress the public key, but it may also be exploited by attackers and thus may lead to some complexity reduction. For example, DOOM-type attacks can use the quasi-cyclic structure to obtain a small speedup factor. Therefore, in security estimation, one usually first evaluates the baseline complexity of (n, k, w) according to generic information set decoding attacks, and then subtracts the corresponding structural discount according to the quasi-cyclic order ℓ . Existing studies usually regard the impact of quasi-cyclic structure as limited for appropriately chosen parameters; it does not fundamentally change the hardness of syndrome decoding.

Definition 4: Public-key indistinguishability problem for quasi-cyclic Goppa codes DIQCG. Let $\mathcal{D}_{\text{QCG}}$ denote the distribution of ℓ -quasi-cyclic Goppa-code public matrices output by the key generation algorithm, namely

$$H_{\text{QCG}} \leftarrow \mathcal{D}_{\text{QCG}}.$$

Let $\mathcal{D}_{\text{rand}}$ denote the distribution of random code matrices with the same size, the same rank condition, and the same systematic ℓ -quasi-cyclic form, namely

$$H_{\text{rand}} \leftarrow \mathcal{D}_{\text{rand}}.$$

The public-key indistinguishability problem for quasi-cyclic Goppa codes asks an attacker, given a matrix M , to decide whether M comes from the real public-key distribution of quasi-cyclic Goppa codes $\mathcal{D}_{\text{QCG}}$, or from the random quasi-cyclic code matrix distribution $\mathcal{D}_{\text{rand}}$. Formally, the distinguishing advantage of an attacker $\mathcal{A}$ is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{DIQCG}} = \left| \Pr_{H_{\text{QCG}} \leftarrow \mathcal{D}_{\text{QCG}}} [\mathcal{A}(H_{\text{QCG}}) = 1] - \Pr_{H_{\text{rand}} \leftarrow \mathcal{D}_{\text{rand}}} [\mathcal{A}(H_{\text{rand}}) = 1] \right|.$$

If the above advantage is negligible for every probabilistic polynomial-time attacker $\mathcal{A}$, then the ℓ -quasi-cyclic Goppa-code public matrices obtained from the key generation algorithm are said to be computationally indistinguishable from random quasi-cyclic code matrices of the same form.

This problem characterizes whether a public key leaks the algebraic structure behind its Goppa code. If an attacker can distinguish a real public matrix from a random quasi-cyclic matrix, then

identifiable structural features remain in the public key; these features may further develop into a structure-recovery attack or key-recovery attack. Therefore, public-key indistinguishability is a key assumption for the structural security of quasi-cyclic Goppa-type code-based cryptography.

In summary, the security of this scheme can be understood at two levels. On one hand, the attacker faces the recovery problem for a low-weight error vector, namely $\text{SD}(n, k, w)$ or $\ell\text{-QCSD}(n, k, w)$ with quasi-cyclic structure. On the other hand, the attacker may also try to identify or recover the hidden twisted Goppa structure from the public matrix, so the scheme also relies on the hardness of private-structure recovery and on the public-key indistinguishability assumption.

6.2 Known Attacks and Practical Security Levels

This proposal uses the widely used recent syndrome decoding estimator `SDEstimator`[16] for parameter-set selection and practical security evaluation of the scheme.¹ The estimator covers mainstream information set decoding (ISD) attack algorithms, including Prange, Stern, Dumer, Ball-Collision, BJMM, BJMM-pdw, May-Ozerov, and Both-May.

The basic goal of these attack algorithms is to recover an error vector e of weight w , given a parity-check matrix H and syndrome $s = He^T$. For a given parameter set, we run `SDEstimator` and take the lowest time complexity among all attack algorithms as the classical security strength under the corresponding parameters.

- The Prange algorithm[34] is the earliest and most basic information set decoding algorithm. Its core idea is to randomly choose an information set and assume that the error vector has no support positions in that information set, so that a candidate error vector can be recovered by linear algebra. If the selected information set happens to avoid all error positions, decoding succeeds. Therefore, the complexity of the Prange algorithm is mainly determined by the probability of randomly selecting an error-free information set. Although the algorithm has a simple structure, it provides the baseline complexity for all subsequent improved ISD attacks.
- The Stern algorithm[35] introduces a collision-search idea on top of the Prange algorithm. It allows a small number of errors in the information set and divides the search space into several parts. Intermediate lists are constructed and matched to find a low-weight error vector satisfying the syndrome condition. Compared with Prange, Stern significantly reduces attack complexity using a time-space tradeoff, but requires additional storage for intermediate lists and matching.
- The Dumer algorithm[15] further improves the blocking and collision strategies used in Stern. Its main idea is to transform the original decoding problem into smaller subproblems by using more refined linear transformations and block processing, and then use list-collision methods to find candidate error vectors. Dumer usually improves the time complexity over Stern and is an important early improvement among efficient ISD attacks.
- The Ball-Collision algorithm[10] is a class of ISD attacks based on the ball-collision idea. Unlike traditional Stern or Dumer methods that use fixed-weight decompositions, the Ball-Collision algorithm searches for candidate vectors within a Hamming ball of a certain radius and uses collision matching to improve the efficiency of low-weight error recovery. This method can be viewed as a further optimization of the list-collision strategy, and it can achieve lower attack complexity than Stern and Dumer in some parameter regions.
- The BJMM algorithm, namely the Becker-Joux-May-Meurer algorithm[4], is an important representative of modern information set decoding attacks. It introduces the representation technique, which allows the same target error vector to be represented in multiple ways as a combination of several lower-weight vectors, thereby increasing the collision success probability and reducing the overall search complexity. BJMM can usually significantly outperform traditional ISD algorithms such as Prange, Stern, and Dumer, and is therefore important in parameter security evaluation for code-based cryptography.
- BJMM-pdw and BJMM-dw are further variants of the BJMM algorithm[4], and are evaluated as BJMM-family variants in `SDEstimator`[16] for parameter-security estimation. Here, pdw usually denotes partial distance weight, meaning that partial-distance or partial-weight constraints are introduced to optimize list construction and collision search; dw is related to the distance-weight

¹The latest implementation code is available at https://github.com/Crypto-TII/syndrome_decoding_estimator.

distribution. The purpose of these variants is to exploit the weight-distribution features of the error vector more finely, thereby further reducing attack complexity in specific parameter ranges. Therefore, BJMM and its variants should be included together in security evaluation.

- The May–Ozerov algorithm[29] is a further improved ISD attack obtained by combining the BJMM idea with nearest-neighbor search techniques. It converts part of the collision-search problem into a nearest-neighbor search problem and reduces time complexity using more efficient data structures and search strategies. Under many code-based cryptographic parameter settings, May–Ozerov often gives the lowest complexity among all classical ISD attacks, so it frequently becomes the strongest attack determining the classical security level of the parameters.
- The Both-May algorithm[12] can be viewed as a further variant of the May family of information set decoding attacks. Its main goal is to obtain a better tradeoff between time complexity and space complexity. The algorithm combines multiple ideas involving list construction, collision search, and nearest-neighbor search, and can approach or outperform May–Ozerov in some parameter regions. Including Both-May therefore makes the security estimate more comprehensive.

In summary, this document does not rely on a single attack algorithm in parameter security evaluation. Instead, it uniformly estimates multiple classical ISD attacks listed above and takes the lowest attack complexity among them as the classical security strength of the parameter set. Let $T_{\mathcal{A}}(n, k, w)$ denote the time complexity of attack algorithm $\mathcal{A}$ under parameter set (n, k, w) . The classical security strength is defined as

$$\lambda_{\text{classical}} = \min_{\mathcal{A}} \log_2 T_{\mathcal{A}}(n, k, w),$$

where $\mathcal{A}$ ranges over information set decoding attack algorithms including Prange, Stern, Dumer, Ball-Collision, BJMM, BJMM-pdw, May–Ozerov, and Both-May. If the possible DOOM speedup brought by quasi-cyclic structure is further considered, then the structural discount

$$\frac{1}{2} \log_2 \ell$$

is subtracted from the classical security strength to obtain the final security level. In the QCTM scheme, we fix $\ell = 19$, and therefore $\frac{1}{2} \log_2(19) \approx 2.12$.

The time-security estimation results for the three submitted parameter sets are as follows.

- **Parameter instance 1: QCTM128**

For the first parameter instance, we substitute code length $n = 10070$, dimension $k = 4957$, quasi-cyclic order $\ell = 19$, and error weight $w = 142$. The resulting security estimate is shown in Table 1.

Table 1: Security-estimation results for the first parameter set

Attack algorithm	Base security strength	Final security strength
BallCollision	155.62 bits	153.50 bits
BJMMdw	153.73 bits	151.61 bits
BJMMpdw	153.71 bits	151.59 bits
BJMM	154.02 bits	151.89 bits
BJMMplus	153.68 bits	151.56 bits
BothMay	153.01 bits	150.88 bits
Dumer	155.60 bits	153.48 bits
MayOzerov	152.99 bits	150.87 bits
Prange	176.14 bits	174.01 bits
Stern	154.64 bits	152.52 bits

Among the attacks considered, **MayOzerov** gives the lowest attack complexity. Therefore, the final security strength of this parameter set is **150.87 bits**, which is above the **128-bit** security level.

- **Parameter instance 2: QCTM256**

For the second parameter instance, we substitute code length $n = 19000$, dimension $k = 9783$, quasi-cyclic order $l = 19$, and error weight $w = 256$. The resulting security estimate is shown in Table 2.

Table 2: Security-estimation results for the second parameter set

Attack algorithm	Base security strength	Final security strength
BallCollision	283.77 bits	281.64 bits
BJMMdw	277.10 bits	274.98 bits
BJMMpdw	277.11 bits	274.98 bits
BJMM	276.50 bits	274.38 bits
BJMMplus	276.94 bits	274.81 bits
BothMay	275.95 bits	273.83 bits
Dumer	283.28 bits	281.16 bits
MayOzerov	275.03 bits	272.90 bits
Prange	308.28 bits	306.15 bits
Stern	282.76 bits	280.64 bits

Among the attacks considered, **MayOzerov** gives the lowest attack complexity. Therefore, the final security strength of this parameter set is **272.90 bits**, which is above the **256-bit** security level.

- **Parameter instance 3: QCTM512**

For the third parameter instance, we substitute code length $n = 38000$, dimension $k = 19891$, quasi-cyclic order $l = 19$, and error weight $w = 503$. The resulting security estimate is shown in Table 3.

Table 3: Security-estimation results for the third parameter set

Attack algorithm	Base security strength	Final security strength
BallCollision	553.96 bits	551.84 bits
BJMMdw	535.02 bits	532.89 bits
BJMMpdw	534.75 bits	532.63 bits
BJMM	532.42 bits	530.30 bits
BJMMplus	534.58 bits	532.46 bits
BothMay	533.88 bits	531.76 bits
Dumer	553.90 bits	551.77 bits
MayOzerov	532.95 bits	530.83 bits
Prange	584.47 bits	582.34 bits
Stern	553.70 bits	551.57 bits

At this large parameter scale, **BJMM** gives the lowest attack complexity. Therefore, the final security strength of this parameter set is **530.30 bits**, which is above the **512-bit** security level.

6.3 Structural Attack

According to the report [2], one of the main reasons for not selecting BIG QUAKE into the 2nd round of NIST-PQC competition is that BIG QUAKE might be insecure due to the structural folding attacks proposed in [19]. To overcome this problem, we aim to evading this kind of structural folding attack by introducing twisted terms into the parity-check matrix. The details are given below.

The QCTM scheme cannot be directly attacked by the folding method of [19]. Let the parity-check matrix of the twisted quasi-cyclic Goppa code $\Gamma(g, L, \eta)$ in QCTM be the following $t \times n$ matrix

over $\mathbb{F}_q$:

$$H_T = \begin{pmatrix} \frac{1}{g(\alpha_0)} & \frac{1}{g(\alpha_1)} & \cdots & \frac{1}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0} & \frac{g(\alpha_1)}{\alpha_1} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}} \\ \frac{g(\alpha_0)}{g(\alpha_0)} & \frac{g(\alpha_1)}{g(\alpha_1)} & \cdots & \frac{g(\alpha_{n-1})}{g(\alpha_{n-1})} \\ \vdots & \vdots & & \vdots \\ \frac{\alpha_0^{t-2}}{g(\alpha_0)} & \frac{\alpha_1^{t-2}}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-2}}{g(\alpha_{n-1})} \\ \frac{g(\alpha_0)}{\alpha_0^{t-1} + \eta\alpha_0^t} & \frac{g(\alpha_1)}{\alpha_1^{t-1} + \eta\alpha_1^t} & \cdots & \frac{g(\alpha_{n-1})}{\alpha_{n-1}^{t-1} + \eta\alpha_{n-1}^t} \\ \frac{\alpha_0^{t-1} + \eta\alpha_0^t}{g(\alpha_0)} & \frac{\alpha_1^{t-1} + \eta\alpha_1^t}{g(\alpha_1)} & \cdots & \frac{\alpha_{n-1}^{t-1} + \eta\alpha_{n-1}^t}{g(\alpha_{n-1})} \end{pmatrix}.$$

It is also the parity-check matrix of the quasi-cyclic twisted GRS code $\mathcal{C} = GRS_{n-t}(g, L, \eta)$. By [22], when $a = \sum_{j=0}^{n-1} \alpha_j \neq 0$, the generator matrix $G_{n-t}(g, L, \eta)$ of this code is

$$\begin{pmatrix} \frac{g(\alpha_0)}{v_0} & \frac{g(\alpha_1)}{v_1} & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \\ \frac{g(\alpha_0)}{v_0} \alpha_0 & \frac{g(\alpha_1)}{v_1} \alpha_1 & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \alpha_{n-1} \\ \vdots & \vdots & & \vdots \\ \frac{g(\alpha_0)}{v_0} \alpha_0^{n-t-2} & \frac{g(\alpha_1)}{v_1} \alpha_1^{n-t-2} & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \alpha_{n-1}^{n-t-2} \\ \frac{g(\alpha_0)}{v_0} \left(\alpha_0^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_0^{n-t} \right) & \frac{g(\alpha_1)}{v_1} \left(\alpha_1^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_1^{n-t} \right) & \cdots & \frac{g(\alpha_{n-1})}{v_{n-1}} \left(\alpha_{n-1}^{n-t-1} + \frac{\eta}{1+a\eta} \alpha_{n-1}^{n-t} \right) \end{pmatrix},$$

where $m(x) = \sum_{j=0}^{n-1} (x - \alpha_j)$ and $v_i = m'(\alpha_i) = \sum_{j=0, j \neq i}^{n-1} (x - \alpha_j)$. When $a = \sum_{j=0}^{n-1} \alpha_j = 0$, $\mathcal{C} = GRS_{n-t}(g, L, \eta)$ is also a twisted GRS code.

Due to the special construction, $GRS_{n-t}(g, L, \eta)$ is a quasi-cyclic twisted GRS code. This code can be folded, but its folded code is neither a GRS code nor a twisted GRS code, because it does not satisfy Theorem 1 of [19]. Therefore, the folding-code method cannot be applied directly as an attack.

More specifically,

$$\Gamma(g, L, \eta) = GRS_{n-t}(g, L, \eta) \cap \mathbb{F}_2^n.$$

It is known that $\Gamma(g, L, \eta)$ has an automorphism σ . Hence, by the property of folded codes,

$$\overline{\Gamma(g, L, \eta)^\sigma} \subset \overline{GRS_{n-t}(g, L, \eta)^\sigma} \cap \mathbb{F}_2^n.$$

In fact, the folded code $\overline{GRS_{n-t}(g, L, \eta)^\sigma}$ is neither a GRS code nor a twisted GRS code, and therefore this code cannot be attacked directly by the folding method.

6.4 Decapsulation Failure Analysis

If Decode succeeds, Encaps and Decaps use the same e and the same C , and therefore obtain the same

$$K = \mathbf{H}(1, e, C).$$

If Decode fails, Decaps uses s from the private key and outputs

$$K = \mathbf{H}(0, s, C).$$

This branch makes the decapsulation interface defined for *every* ciphertext and *is consistent* with the KEM behavior of the submitted implementation. Therefore, QCTM has *no decapsulation failure*.

7 Reference and Optimizational Implementations

7.1 Instantiation of the Irreducible Polynomials $f(z)$ and $F(y)$

Considering the structural design requirements of the quasi-cyclic twisted Goppa codes needed by QCTM, the two irreducible polynomials

$$f(z) \in \mathbb{F}_2[z] \text{ and } F(y) \in \mathbb{F}_q[y]$$

must be adapted to the concrete parameter instances, where $\mathbb{F}_q \triangleq \mathbb{F}_2[z]/(f(z))$. Also considering implementation efficiency, we choose the following irreducible polynomials for the three parameter sets.

Instance	m	n	ℓ	t_0	t	$f(z)$	$F(y)$
QCTM128	18	10070	19	15	285	$z^{18} + z^3 + 1$	$y^{15} + y^7 + z^7$
QCTM256	18	19000	19	27	513	$z^{18} + z^7 + 1$	$y^{27} + z$
QCTM512	18	38000	19	53	1007	$z^{18} + z^3 + 1$	$y^{53} + y^6 + z$

7.2 Matrix Validation

The MatGen implementation validates the following conditions:

- the number of rows of $\hat{H}_T$ is $u = m(t - 1) + 1$;
- $r_0 = u \bmod \ell$, and T_2 contains r_0 rows;
- the right-side width of T_1 is $n - (u - r_0) = n - u + r_0$, not $n - u$;
- the public part of T_1 satisfies ℓ -block-circulant consistency.

7.3 Reference Implementation

The submission package provides a C reference implementation in the directory

Implementations/Reference_Implementation.

The implementation provides the `crypto_kem_keypair`, `crypto_kem_enc`, and `crypto_kem_dec` interfaces, and also provides the commercial cryptographic algorithm submission API adaptation files `KEM_AlgorithmInstance.h` and `KEM_AlgorithmInstance.c`.

7.4 Optimizational Implementation

The submission package will provide a C optimized implementation in the directory

Implementations/Optimized_Implementation.

The optimized implementation keeps the same `crypto_kem_keypair`, `crypto_kem_enc`, and `crypto_kem_dec` interfaces as the reference implementation, but reduces repeated work in the main hot paths. In particular, encapsulation uses a cached public-key column representation, while decapsulation reuses a decoded secret-key/Goppa structure instead of rebuilding it for every call. The optimized implementation contains the three parameter-set directories QCTM128, QCTM256, and QCTM512.

8 Performance and Sizes

8.1 Algorithm Performance (Thousands of Clock Cycles)

The experiments were conducted on an MSI GP66 Leopard 11UG machine equipped with an Intel Core i7-11800H @ 2.30GHz processor, with 8 physical cores and 16 logical threads, and 32 GiB of physical memory. The host system was 64-bit Microsoft Windows 11 Home Chinese Edition (version 10.0.22631); the tests were run under Ubuntu 24.04 LTS on WSL2, with Linux kernel version 6.18.33.2-microsoft-standard-WSL2 and amd64 as the target architecture. The performance evaluation used SUPERCOP 20220506 and GCC 13.3.0, with the main compilation options `-march=native -mtune=native -O3 -fomit-frame-pointer -fwrapv -fPIC -fPIE -fcommon`. SHAKE256 was implemented using the `libkeccak.a` library built in the SUPERCOP build directory and invoked through the interface provided by `libkeccak.a.headers/SimpleFIPS202.h`; in the generated binary, `FIPS202_SHAKE256` wraps and calls `SHAKE256`. This choice is consistent with the SHAKE256 implementation used in the previous reproduction of Classic McEliece. The cycle-counting implementation recorded by SUPERCOP was `amd64rdpmc`, and the processor frequency was 2,303,999,000 cycles/s.

Parameter set	Key generation	Encapsulation	Decapsulation
QCTM128	11,795,687.2	183.4	3,063.1
QCTM256	62,776,212.1	425.2	10,593.5
QCTM512	880,022,864.4	3,326.9	54,024.2

The values in the table are measured in thousands of clock cycles and are taken from the `avg_kcycles` output of SUPERCOP. For each parameter set, key generation, encapsulation, and decapsulation each contain 100 raw clock-cycle samples.

8.2 Parameter Sizes (Bytes)

Instance	Public-key bytes	Private-key bytes	Ciphertext bytes	Shared-key bytes
QCTM128	167,987	192,545	640	32
QCTM256	595,662	641,942	1,153	32
QCTM512	2,374,727	2,467,243	2,264	64

8.3 Known Answer Tests

The submission package provides KAT files for the three instances:

- the KAT file for the first parameter instance, QCTM128;
- the KAT file for the second parameter instance, QCTM256;
- the KAT file for the third parameter instance, QCTM512.

Each KAT file contains sample public keys, private keys, ciphertexts, and shared keys, and can be regenerated by the corresponding implementation.

9 Innovation Statement

The main innovations of this scheme are as follows:

1. Twisted terms for resisting the known structural folding attacks. The underlying code family is quasi-cyclic twisted Goppa codes[36], and one of its advantages lies in that the introducing of twisted terms breaks the condition for launching the known structural folding attacks due to [19].

2. Local quasi-cyclicity. The public-key matrix is a hybrid systematic matrix consisting of a local quasi-cyclic main part and ordinary tail rows. This scheme does not require all $m(t-1)+1$ rows of $\tilde{H}_t$ to support quasi-cyclic compression. Systematic reduction only requires the first $u-r_0$ rows to form the ℓ -block-circulant main part T_1 , and stores the remaining r_0 rows as ordinary tail rows T_2 . This design avoids conflating ordinary binary rank with block-compatible rank, and it also avoids incorrectly claiming that the whole block matrix can be compressed in a purely quasi-cyclic way.

3. Decoding algorithm for twisted Goppa codes[36, 37]. We introduce twisted Goppa codes, which generalize classical Goppa codes by adding twisted terms, and have their efficient decoding algorithm.

The design of the QCTM scheme draws on Classic McEliece[11] and BIG QUAKE[1]. Compared with Classic McEliece, the overall public-key size is greatly reduced. Compared with BIG QUAKE, the public-key size of the QCTM scheme increases slightly, but security is enhanced. The intended effect is to strike a balance between security and efficiency, especially public-key size. Overall, the security of the QCTM scheme lies between that of Classic McEliece and BIG QUAKE.

10 Conclusion

QCTM uses $m(t-1)+1$ rows of twisted Goppa parity-check structure, a local quasi-cyclic main part T_1 , and ordinary tail rows T_2 to form a hybrid public matrix. The public key is stored as $(\psi(T_1), T_2)$, and the private key is stored as (δ, Γ', s) . During decapsulation, Γ' is used to temporarily

recompute the auxiliary matrices needed for decoding. This specification is consistent with the three submitted parameter sets, matrix shapes, public-key size formula, and KEM interfaces in the submitted implementation.

References

- [1] M. Bardet, E. Barelli, O. Blazy, et al., “BIG QUAKE: BInary Goppa QUAsi-cyclic Key Encapsulation,” NIST Post-Quantum Cryptography Round 1 submission, 2017. NIST Round 1 submissions.
- [2] R. Perlner, “Code based Crypto in the NIST competition: PQC Standardization Process,” NIST presentation slides, May 2019.
- [3] M. Bardet, R. Mora, and J.-P. Tillich, “Polynomial Time Key-Recovery Attack on High Rate Random Alternant Codes,” *IEEE Transactions on Information Theory*, vol. 70, no. 6, pp. 4492–4511, Jun. 2024.
- [4] A. Becker, A. Joux, A. May, and A. Meurer, “Decoding random binary linear codes in $2^{n/20}$: how $1 + 1 = 0$ improves information set decoding,” in *Advances in Cryptology – EUROCRYPT 2012*, LNCS 7237, pp. 520–536, Springer, 2012.
- [5] T. P. Berger, “Cyclic alternant codes induced by an automorphism of a GRS code,” in *Finite Field Theory, Applications and Algorithm*, Contemporary Mathematics, vol. 225, pp. 143–154, American Mathematical Society, 1999.
- [6] T. P. Berger, “New classes of cyclic extended Goppa codes,” *IEEE Transactions on Information Theory*, vol. 45, no. 4, pp. 1264–1266, 1999.
- [7] T. P. Berger, “Goppa and related codes invariant under a prescribed permutation,” *IEEE Transactions on Information Theory*, vol. 46, no. 7, pp. 2628–2633, 2000.
- [8] T. P. Berger, “Quasi-cyclic Goppa codes,” in *Proceedings of the IEEE International Symposium on Information Theory*, 2000.
- [9] E. Berlekamp, “Goppa codes,” *IEEE Transactions on Information Theory*, vol. 19, no. 5, pp. 590–592, 1973.
- [10] D. J. Bernstein, T. Lange, and C. Peters, “Smaller decoding exponents: ball-collision decoding,” in *Advances in Cryptology – CRYPTO 2011*, LNCS 6841, pp. 743–760, Springer, 2011.
- [11] D. J. Bernstein, T. Chou, C. Cid, et al., “Classic McEliece: conservative code-based cryptography,” NIST Post-Quantum Cryptography Round 4 submission, 2025. NIST Round 4 submissions.
- [12] L. Both and A. May, “Optimizing BJMM with nearest neighbors: full decoding in $2^{2n/21}$ and McEliece security,” in *Proceedings of the Tenth International Workshop on Coding and Cryptography (WCC 2017)*, 2017.
- [13] A. Couvreur, I. M. Corbella, and R. Pellikaan, “A polynomial time attack against algebraic geometry code based public key cryptosystems,” in *Proceedings of the IEEE International Symposium on Information Theory*, pp. 1446–1450, 2014.
- [14] A. Couvreur, R. Mora, and J.-P. Tillich, “A new approach based on quadratic forms to attack the McEliece cryptosystem,” in J. Guo and R. Steinfield (eds.), *Advances in Cryptology – ASIACRYPT 2023*, LNCS 14441, pp. 3–38, Springer, 2023.
- [15] I. Dumer, “On minimum distance decoding of linear codes,” in *Proceedings of the 5th Joint Soviet-Swedish International Workshop on Information Theory*, Moscow, pp. 50–52, 1991.
- [16] A. Esser and E. Bellini, “Syndrome Decoding Estimator,” in G. Hanaoka et al. (eds.), *Public-Key Cryptography – PKC 2022*, LNCS 13177, pp. 112–141, Springer, 2022.

- [17] J.-C. Faugère, A. Otmani, L. Perret, and J.-P. Tillich, “Algebraic Cryptanalysis of McEliece Variants with Compact Keys,” in H. Gilbert (ed.), *Advances in Cryptology – EUROCRYPT 2010*, LNCS 6110, pp. 279–298, Springer, 2010.
- [18] J.-C. Faugère, L. Perret, and F. de Portzamparc, “Algebraic attack against variants of McEliece with Goppa polynomial of a special form,” in *Advances in Cryptology – ASIACRYPT 2014*, pp. 21–41, Springer, 2014.
- [19] J.-C. Faugère, A. Otmani, L. Perret, F. de Portzamparc, and J.-P. Tillich, “Folding Alternant and Goppa Codes With Non-Trivial Automorphism Groups,” *IEEE Transactions on Information Theory*, vol. 62, no. 1, pp. 184–198, Jan. 2016.
- [20] V. Goppa, “A new class of linear error correcting codes,” *Problems of Information Transmission*, vol. 6, no. 3, pp. 207–212, 1970.
- [21] V. Goppa, “Rational representation of codes and (L, g) codes,” *Problems of Information Transmission*, vol. 7, no. 3, pp. 41–49, 1971.
- [22] D. Huang, Q. Yue, Y. Niu, and X. Li, “MDS or NMDS self-dual codes from twisted generalized Reed–Solomon codes,” *Designs, Codes and Cryptography*, vol. 89, no. 9, pp. 2195–2209, 2021. doi: <https://doi.org/10.1007/s10623-021-00910-7>.
- [23] W. Huffman and V. Pless, *Fundamentals of Error-Correcting Codes*. Cambridge University Press, 2003.
- [24] X. Jia, F. Li, H. Sun, Q. Yue, “Generator Polynomials of Cyclic Expurgated or Extended Goppa Codes,” *IEEE Trans. Inf. Theory*, vol. 70, no.11, pp. 7711–7722, 2024.
- [25] F. Li, X. Jia, H. Sun, Q. Yue, “Quasi-cyclic binary extended and expurgated Goppa codes and their parameters,” *Des. Codes Cryptogr.* vol. 94, no. 4, pp. 85, 2026.
- [26] X. Li and Q. Yue, “Non-binary irreducible quasi-cyclic parity-check subcodes of Goppa codes and extended Goppa codes,” *Designs, Codes and Cryptography*, vol. 90, no. 7, pp. 1629–1647, 2022.
- [27] X. Li and Q. Yue, “Construction of expurgated and extended Goppa codes with dihedral automorphism groups,” *IEEE Transactions on Information Theory*, vol. 68, no. 10, pp. 6472–6480, 2022.
- [28] R. Lidl and H. Niederreiter, *Finite Fields*. Cambridge University Press, 2008.
- [29] A. May and I. Ozerov, “On computing nearest neighbors with applications to decoding of binary linear codes,” in *Advances in Cryptology – EUROCRYPT 2015*, LNCS 9056, pp. 203–228, Springer, 2015.
- [30] R. J. McEliece, “A public-key cryptosystem based on algebraic coding theory,” DSN Progress Report 42-44, pp. 114–116, Jet Propulsion Laboratory, 1978.
- [31] R. Misoczki and P. S. L. M. Barreto, “Compact McEliece Keys from Goppa Codes,” in M. J. Jacobson Jr., V. Rijmen, and R. Safavi-Naini (eds.), *Selected Areas in Cryptography – SAC 2009*, LNCS 5867, pp. 376–392, Springer, 2009.
- [32] H. Niederreiter, “Knapsack-type cryptosystems and algebraic coding theory,” *Problems of Control and Information Theory*, vol. 15, no. 2, pp. 159–166, 1986.
- [33] N. J. Patterson, “The algebraic decoding of Goppa codes,” *IEEE Transactions on Information Theory*, vol. 21, no. 2, pp. 203–207, 1975.
- [34] E. Prange, “The use of information sets in decoding cyclic codes,” *IRE Transactions on Information Theory*, vol. 8, no. 5, pp. 5–9, 1962.
- [35] J. Stern, “A method for finding codewords of small weight,” in G. D. Cohen and J. Wolfmann (eds.), *Coding Theory and Applications*, LNCS 388, pp. 106–113, Springer, 1989.

- [36] J. Sui and Q. Yue, “Twisted Goppa codes with an efficient decoding algorithm and quasi-cyclic properties,” *IEEE Transactions on Information Theory*, vol. 69, no. 9, pp. 5660–5669, 2023.
- [37] H. Sun, Q. Yue, X. Jia, and C. Li, “Decoding algorithms of twisted GRS codes and twisted Goppa codes,” *IEEE Transactions on Information Theory*, vol. 71, no. 2, pp. 1018–1027, 2025.
- [38] C. Wieschebrink, “Cryptanalysis of the Niederreiter public key scheme based on GRS subcodes,” in *Post-Quantum Cryptography*, pp. 61–72, Springer, 2010.